- **数组**
  - 语法

```
const arr1: number[] = [1, 2, 3]
const arr2: string[] = ['a', 'b', 'c']
const arr3: (number | string)[] = [1, 2, 3, 'a', 'b', 'c'] // “|”是联合类型语法，表示这几种类型都可以
// 或
const arr4: Array<number> = [1, 2, 3] // 采用泛型来声明数组，其中“<>”是泛型语法
```

  - 通过索引，取数组类型中元素的类型

```
type IArr = ["a", "b", "c"]

type T1 = IArr[0] // "a"
type T2 = IArr[1] // "b"
```

  - 通过number，取数组中元素的联合类型

```
type IArr = ["a", "b", "c"]

type T = IArr[number] // "a" | "b" | "c"
```

  - 通过length属性，取数组的长度

```
type IArr = ["a", "b", "c"]

type T = IArr["length"] //  3
```

- **元组（长度固定，数组中的元素有多种类型）**
  - 语法

```
const user: [string, number, boolean] = ['Sun', 20, true]
```

```
// 或
const user2: [name: string, age: number, male: boolean] = ['Sun', 20, true] // 可以标识一下每一项数据代表什么
user[3] // 报错：长度为 "3" 的元组类型 "[string, number, boolean]" 在索引 "3" 处没有元素。
```

- 将元组包装成新的元组
  - 基础用法

```
type ITuple = [number, boolean, string]

type DataList = { [key in keyof ITuple]: { data: ITuple[key] } }

let dataList!: DataList

dataList[0].data // number
dataList[1].data // boolean
dataList[2].data // string
```

  - 使用泛型
    - 只使用泛型推断功能，类型错误（成所有元素的联合类型了）

```
type Fn = <T extends unknown[]>(
  data: T
) => {
  [key in keyof T]: { data: T[key] }
}

let fn!: Fn

const res = fn([123, true, 'abc']) // 只利用泛型推断

res[0].data // string | number | boolean
res[1].data // string | number | boolean
res[2].data // string | number | boolean
```

    - 泛型约束联合空数组（需要泛型直接推断元组的类型），类型正确（元素对应自己的类型）
      - 正确示例

```
type Fn = <T extends unknown[] | []>( // 泛型约束联合空数组（泛型直接推断元组的类型）
  data: T
) => {
  [key in keyof T]: { data: T[key] }
}

let fn!: Fn

const res = fn([123, true, 'abc'])

res[0].data // number
res[1].data // boolean
res[2].data // string
```

- 错误示例

```
type Fn = <T extends unknown[] | []>(data: { // 泛型约束联合空数组（泛型【没有】直接推断元组的类型）
  [key in keyof T]: { data: T[key] }
}) => T

let fn!: Fn

const res = fn([{ data: 123 }, { data: true }, { data: 'abc' }])

res[0] // string | number | boolean
res[1] // string | number | boolean
res[2] // string | number | boolean
```

- 传入泛型，类型正确（元素对应自己的类型）

```
type Fn = <T extends unknown[]>(
  data: T
) => {
  [key in keyof T]: { data: T[key] }
}

```

```
let fn!: Fn

const res = fn<[number, boolean, string]>([123, true, 'abc']) // 传入泛型

res[0].data // number
res[1].data // boolean
res[2].data // string
```

- **对象**
  - 常规属性

```
type User = {
  name: string
  age: number
}

const user: User = {
  name: 'Sun',
  age: 20
}
```

  - 可选属性

```
type User = {
  name: string
  age: number
  motorbike?: string
}

const user1: User = {
  name: 'Sun',
  age: 20
}
const user2: User = {
  name: 'Sun',
  age: 20,
  motorbike: 'haojue'
```

```
}
```

- 任意属性

```
type User = {
  name: string
  age: number
  [key: string]: any
}

const user: User = {
  name: 'Sun',
  age: 20,
  qq: 790964731,
  weixin: 'sjf_08'
}
```

  - 键可以写成任意内容，但习惯用key
  - 键的类型可以是string、number、symbol
  - 值的类型必须包含已有属性的类型
- 映射属性

```
// 示例1
type User = {
  [key in 'name' | 'age']: string
}

const user: User = {
  name: 'Sun',
  age: '20',
  qq: '790964731' // 无法添加qq属性
}

// 示例2
type Compute<T extends object> = {
  [key in keyof T]: T[key]
}
```

    - 键可以写成任意内容，但习惯用key
    - 如果只有一个映射属性声明，可以用工具方法Record替代，如Record<'name' | 'age', string>
  - 映射属性别名

```
type Compute<T extends object> = {
  [key in keyof T as `key_${key & string}`]: T[key]
}

type Obj = {
  a: string
  b: number
  c: boolean
}

type IT = Compute<Obj> // type IT = { key_a: string; key_b: number; key_c: boolean; }
```

  - 只读属性

```
type User = {
  readonly name: string
  age: number
}

const user: User = {
  name: 'Sun',
  age: 20,
}
user.name = 'zhangsan' // 报错：无法为“name”赋值，因为它是只读属性。
```

  - 索引属性

```
type IArr = {
  [key: number]: any
}

const arr: IArr = [1, '123']
const obj: IArr = {
```

```
  0: 100,
  '1': 100, // 自动转化
  2: 200
}
```

- 通过索引获取值的类型

```
type IPerson = {
  name: string,
  address: {
    num: number
  }
}
type IPersonName = IPerson['name'] // type IPersonName = string
type IPersonAddressNum = IPerson['address']['num'] // type IPersonAddressNum = number
```

- 取对象的所有属性或值的联合类型

```
type IPerson = {
  name: string,
  address: {
    num: number
  }
}
type IPropTypeUnion = keyof IPerson // type IPropTypeUnion = 'name' | 'address'
type IValueTypeUnion = IPerson[keyof IPerson] // type IValueTypeUnion = string | { num: number; }
```

- 值的类型为函数
  - 有逆变效果

```
type I_Obj = {
  fn: () => number;
  count: number;
};

```

```
const fn: I_Obj = {
  fn: () => 1,
  count: 1,
};
```

- 无逆变效果（容易混淆）

```
type I_Obj = {
  fn(): number;
  count: number;
};

const fn: I_Obj = {
  fn: () => 1,
  count: 1,
};

// -------------------- 下面的写法是带有属性的函数 --------------------

    type I_Fn = {
      (): number;
      count: number;
    };

const fn: I_Fn = () => 1;
fn.count = 1;
```

- **函数**
  - 类型声明
    - 写在函数声明里

```
function sum(a: number, b:number): number {
  return a + b
}
sum(1, 2)
```

- 使用类型别名

```
type Sum = (a: number, b:number) => number
const sum2: Sum = (a, b) => a + b
sum2(1, 2)
```

- 可选参数

```
// 如果有其他必选参数，必选参数必须放在可选参数之前
function sum(a: number, b?:number): number {
  return a + (b ?? 0)
}
// 或
function sum(a: number, b:number = 0): number {
  return a + b
}
sum(1, 2)
sum(1)
```

- 剩余参数

```
function sum(a: number, ...args: number[]){
  return args.reduce((mome, current)=> mome + current, a)
}
sum(1, 2, 3, 4)
```

- 为this标注类型

```
// this类型标注必须写在参数的最前面
function getName(this: typeof person, key: keyof typeof person){
  return this[key]
}

const person = {
  name: 'Sun',
  age: 20
}
```

```
getName.call(person, 'name')
```

- 调用签名（带有属性的函数）

```
type I_Counter = {
  (num: number): number
  count: number
}

const counter: I_Counter = num => num++ // 需要用const声明
counter.count = 100
```

- 构造签名

```
type SomeConstructor = {
  new (s: string): SomeObject;
};
function fn(ctor: SomeConstructor) {
  return new ctor("hello");
}
```

- **枚举**
  - 普通枚举（编译成js后会生成同名对象）

```
enum USER_ROLE {
  USER = 'a',
  ADMIN = 10,
  SUPER_ADMIN // 11，不写值可以自动推导为前面的值+1，如果从第一个元素就开始不写值那就从0开始自动赋值
}

USER_ROLE.USER // 'a'
USER_ROLE[10] // ADMIN，值为数字的时候可以通过反举来取到键
```

  - 常量枚举（前面加const，编译成js后不会生成同名对象，不能反举）

```
const enum USER_ROLE {
  USER, // 0
  ADMIN // 1
}

USER_ROLE.USER // 0
```